

Understanding Floorplan of SPM design using OpenLane

INTRODUCTION:

The Floorplan Lab in OpenLane focuses on organizing the physical layout of a digital design after synthesis by defining the chip's core area, aspect ratio, and utilization using user-specified parameters. It includes the placement of I/O pins, power rings, and standard cell rows to ensure efficient area usage and proper power distribution. OpenLane automates the generation of a routable and optimized floorplan through simple commands. This floorplan forms the foundation for subsequent placement, clock tree synthesis, and routing stages.

APPLICATION:

The Floorplan Lab in OpenLane is primarily used to define and optimize the physical layout foundation of a digital ASIC design. It helps students, researchers, and designers understand how core area selection, utilization, and pin placement impact routability, timing, and power distribution. The lab is widely applicable in educational ASIC flows, RISC-V and SoC designs, macro integration, and early-stage prototyping using open-source PDKs. Proper floorplan enables efficient placement and routing in subsequent stages, reducing design iterations and physical design issues.

OBJECTIVE:

After completing this lab, you will be able to:

- Understand the role of floorplan in the RTL-to-GDSII flow
- Configure core area, utilization, and aspect ratio parameters in OpenLane
- Place I/O pins, power rings, and standard cell rows effectively
- Analyse floorplan results to identify congestion and area issues
- Prepare a design for successful placement, CTS, and routing stages

PROCEDURE TO CREATE A LAB:

Step 1: Set up OpenLane flow

1.1: Invoke OpenLane in the terminal using the command: `openlane`

```
coe-4@ip-10-100-19-29:~/OpenLaneUser/designs$ openlane
=====
OpenLane started for user: coe-4
Workspace : /home/coe-4/OpenLaneUser
PDK Root  : /labroot/openlane_pdk
=====
OpenLane Container (ff5509f):/openlane$
```

1.2: Open folder- designs: `cd designs`

Step 2: Set up Project Directory

2.1: Create a directory for your lab: `mkdir <project_name>`

2.2: Open lab directory: `cd <project_name>`

2.3: Create directory for project: `mkdir spm`

2.4: Open project directory: `cd spm`

2.5: Inside project directory, create folder: `mkdir src`

2.6: Open folder: `cd src`

Step 3: Create Design file

3.1: Open vi editor for design with command: `vi spm.v`

RTL Design Code:

```
module spm(clk, rst, x, y, p);
    parameter size = 32;
    input clk, rst;
    input y;
    input[size-1:0] x;
    output p;
    wire[size-1:1] pp;
    wire[size-1:0] xy;

    genvar i;

    CSADD csa0 (.clk(clk), .rst(rst), .x(x[0]&y), .y(pp[1]),
    .sum(p));

    generate for(i=1; i<size-1; i=i+1) begin
```

```
        CSADD csa (.clk(clk), .rst(rst), .x(x[i]&y),
.y(pp[i+1]), .sum(pp[i]));

    end endgenerate

    TCMP tcmp (.clk(clk), .rst(rst), .a(x[size-1]&y),
.s(pp[size-1]));
endmodule

module TCMP(clk, rst, a, s);
    input clk, rst;
    input a;
    output reg s;
    reg z;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            //Reset logic goes here.
            s <= 1'b0;
            z <= 1'b0;
        end
        else begin
            //Sequential logic goes here.
            z <= a | z;
            s <= a ^ z;
        end
    end
end

endmodule

module CSADD(clk, rst, x, y, sum);
    input clk, rst;
    input x, y;
    output reg sum;
    reg sc;
    // Half Adders logic
```

```
wire hsum1, hco1;

assign hsum1 = y ^ sc;
assign hco1 = y & sc;

wire hsum2, hco2;

assign hsum2 = x ^ hsum1;
assign hco2 = x & hsum1;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        //Reset logic goes here.
        sum <= 1'b0;
        sc <= 1'b0;
    end
    else begin
        //Sequential logic goes here.
        sum <= hsum2;
        sc <= hco1 ^ hco2;
    end
end

endmodule
```

3.2: save the file using command- :wq

Step 4: Create .json file

4.1: Open vi editor: vi config.json

```
{
    "DESIGN_NAME": "spm",
    "VERILOG_FILES": "dir::src/*.v",
    "CLOCK_PERIOD": 10,
    "CLOCK_PORT": "clk",
    "CLOCK_NET": "ref::$CLOCK_PORT",
    "FP_PDN_VOFFSET": 7,
    "FP_PDN_HOFFSET": 7,
```

```
"FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
"FP_PDN_SKIPTRIM": true,
"pdk::sky130*": {
    "FP_CORE_UTIL": 45,
    "scl::sky130_fd_sc_hd": {
        "CLOCK_PERIOD": 10
        "FP_SIZING" : "absolute",
        "DIE_AREA" : "0.0 0.0 101.85 112.57",
        "CORE_AREA" : "5.52 10.88 96.14 100.64",
    },
    "scl::sky130_fd_sc_hd11": {
        "CLOCK_PERIOD": 10
    },
    "scl::sky130_fd_sc_hs": {
        "CLOCK_PERIOD": 8
    },
    "scl::sky130_fd_sc_ls": {
        "CLOCK_PERIOD": 10,
        "MAX_FANOUT_CONSTRAINT": 5
    },
    "scl::sky130_fd_sc_ms": {
        "CLOCK_PERIOD": 10
    }
},
"pdk::gf180mcu*": {
    "CLOCK_PERIOD": 24.0,
    "FP_CORE_UTIL": 40,
    "MAX_FANOUT_CONSTRAINT": 4,
    "PL_TARGET_DENSITY": 0.5
}
}
```

Note : To write the spm.sdc file, refer to Module 4 and place spm.sdc inside the spm directory.

Step 5: Pin_order.cfg file

```
#N
@min_distance=0.1
x.*
#S
$1
rst
#E
clk
#W
p
y
```

pin_order.cfg is used during floorplan in OpenLane to control the side and order of I/O pin placement around the chip boundary

Step 6: Use the following commands to run the flow:

OpenLane to start the design flow in interactive mode

```
i) flow.tcl -interactive
```

Used in the OpenLane interactive shell to prepare your design for the digital IC design flow.

```
ii) prep -design spm
```

Runs yosys synthesis on the current design.

```
iii) run_synthesis
```

To initiate floorplan

```
iv) run_floorplan
```

```
[INFO]: Using configuration in 'designs/spm/config.json'...
[INFO]: PDK Root: /openlane/pdks
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/spm/runs/RUN_2026.01.23_05.16.33
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
% run_synthesis
[STEP 1]
[INFO]: Running Synthesis (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/synthesis/2-sta.log)...
% run_floorplan
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/floorplan/3-initial_fp.log)...
[WARNING]: Current core area is too small for the power grid settings chosen. The power grid will be scaled down.
[INFO]: Floorplanned with width 90.62 and height 89.76.
[STEP 4]
[INFO]: Running IO Placement (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/floorplan/4-place_io.log)...
[STEP 5]
[INFO]: Running Tap/Decap Insertion (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/floorplan/5-tap.log)...
[INFO]: Power planning with power {VPWR} and ground {VGND}...
[STEP 6]
[INFO]: Generating PDN (log: designs/spm/runs/RUN_2026.01.23_05.16.33/logs/floorplan/6-pdn.log)...
%
```

Step 7: View Floorplan using klayout:

Locate the DEF file

i) designs/spm/runs/RUN_2026.01.23_05.16.33/results/floorplan/spm.def

Open in klayout in the same directory where spm.def is located

ii) klayout spm.def

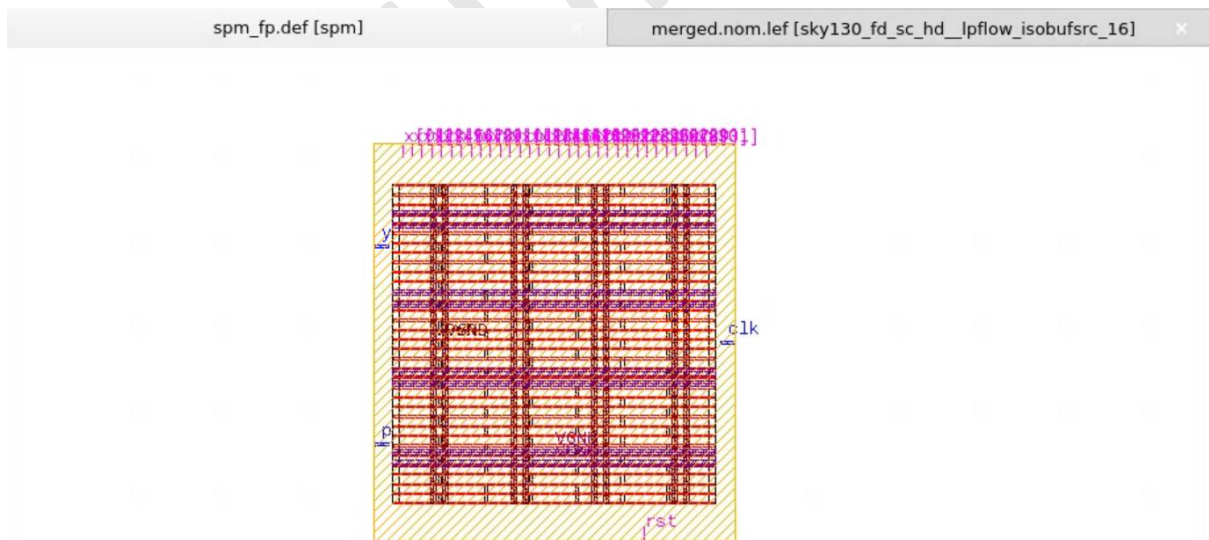


Figure 1: klayout view of floorplan.def

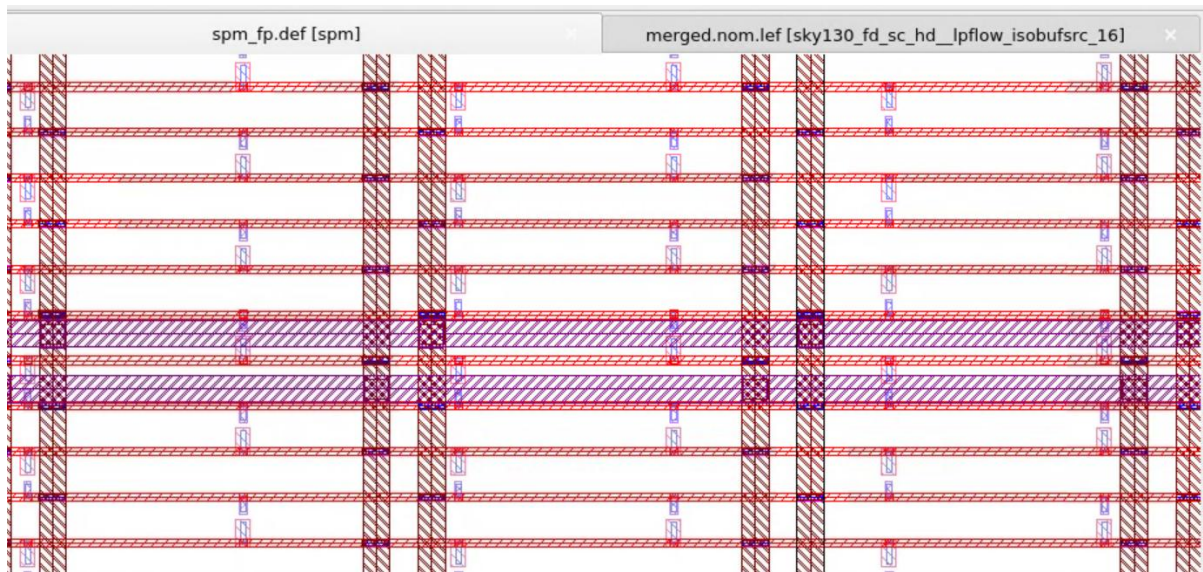


Figure 2: Zoomed klayout view of floorplan.def

OBSERVATION:

- Die and core dimensions defined by the floorplan
- Core utilization percentage and available routing space
- I/O pin placement around the core boundary
- Power distribution network (PDN) structure and pitch
- Standard cell row generation and alignment
- Floorplan DEF and layout visualization in klayout

OUTPUT:

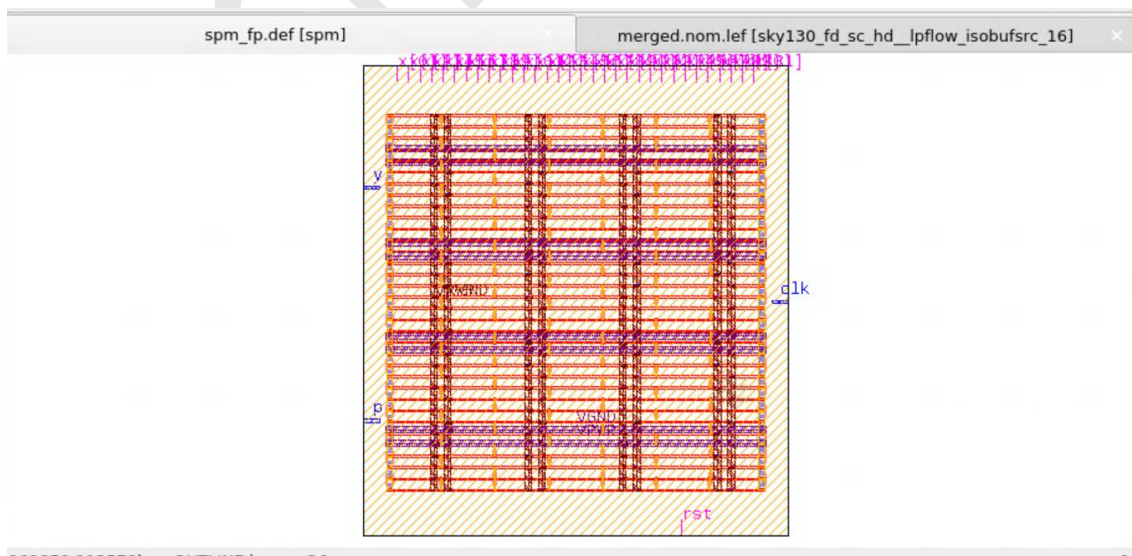


Figure 3: Output of floorplan.def

Output of the floorplan stage is a floorplan.def that shows:

- Die area & core area
- Standard-cell rows
- Power distribution network (VDD/VSS rails & straps)
- I/O pin placement

CONCLUSION:

In this floorplan lab, OpenLane was successfully used to generate a valid and optimized floorplan for the given RTL design. The floorplanning stage correctly defined the die and core areas, established standard cell rows, placed I/O pins as per configuration, and implemented a robust power distribution network (PDN). The generated floorplan DEF was verified using layout viewers, confirming proper core utilization, power grid structure, and pin placement without errors. This validated floorplan provides a strong foundation for subsequent placement, clock tree synthesis, and routing stages, ensuring smooth progression toward a manufacturable ASIC design.